

BlazeDemo Automation Framework – Source Code Document

1. Introduction

Project Name

BlazeDemo Flight Booking Automation Framework

Technology Stack

- Java
- Selenium WebDriver
- TestNG
- Maven
- Apache POI
- Extent Reports
- Jenkins
- GitHub

2. Project Structure

BlazeDemoAutomation

```
|— src/test/java
|   |— base
|   |— pages
|   |— testcases
|   |— defecttests
|   |— listeners
|   |— utilities
|   └— dataproviders
|
|— src/test/resources
|   └— config.properties
|
|— testdata
|   └— BookingData.xlsx
|
|— reports
|— screenshots
|— drivers
|
|— testng.xml
|— crossbrowser.xml
|— defect-testng.xml
|— pom.xml
└— Jenkinsfile
```

- ▼  BlazeDemoAutomation
 -  src/main/java
 -  src/main/resources
 - ▼  src/test/java
 - ▼  base
 - >  BaseTest.java
 - ▼  dataproviders
 - >  BookingDataProvider.java
 - ▼  defecttests
 - >  EmptyCardNumberTest.java
 - >  EmptyPassengerDetailsTest.java
 - >  ExpiredCardTest.java
 - ▼  listeners
 - >  TestListener.java
 - ▼  pages
 - >  ConfirmationPage.java
 - >  FlightSelectionPage.java
 - >  HomePage.java
 - >  PurchasePage.java
 - ▼  testcases
 - >  CrossBrowserLaunchTest.java
 - >  EndToEndBookingTest.java
 - >  LaunchTest.java
 - >  SearchFlightTest.java
 - ▼  utilities
 - >  ConfigReader.java
 - >  DriverFactory.java
 - >  ExcelUtils.java
 - >  ExtentManager.java
 - >  ScreenshotUtil.java
 - >  WaitUtils.java
 - ▼  src/test/resources
 -  config.properties
 - >  JRE System Library [JavaSE-21]
 - >  Maven Dependencies
 - >  drivers
 - >  reports
 - >  screenshots
 - >  src
 - >  target
 - ▼  testdata
 -  BookingData.xlsx
 - >  test-output
 -  crossbrowser.xml
 -  defect-testng.xml
 -  Dockerfile
 -  Jenkinsfile
 -  pom.xml
 -  testng.xml

3. Base Package

3.1 BaseTest.java

```
package base;

import org.openqa.selenium.WebDriver;

import org.testng.annotations.AfterMethod;

import org.testng.annotations.BeforeMethod;

import org.testng.annotations.Listeners;

import org.testng.annotations.Optional;

import org.testng.annotations.Parameters;

import listeners.TestListener;

import utilities.ConfigReader;

import utilities.DriverFactory;

@Listeners(TestListener.class)

public class BaseTest

{

    protected WebDriver driver;

    @BeforeMethod

    @Parameters("browser")

    public void setup(@Optional("chrome") String browser)

    {

        driver = DriverFactory.initializeDriver(browser);

        driver.get(ConfigReader.getProperty("url"));

    }

}
```

```
@AfterMethod

public void tearDown()

{

    if (driver != null) {

        driver.quit();

    }

}

}
```

4. Data Providers Package

4.1 BookingDataProvider.java

```
package dataproviders;

import org.testng.annotations.DataProvider;

import utilities.ExcelUtils;

public class BookingDataProvider

{

    @DataProvider(name = "passengerData")

    public Object[][] getPassengerData()

    {

        return ExcelUtils.getTestData("PassengerData");

    }

}
```

5. Defect Tests Package

5.1 EmptyCardNumberTest.java

```
package defecttests;

import org.testng.Assert;

import org.testng.annotations.Test;

import base.BaseTest;

import pages.ConfirmationPage;

import pages.FlightSelectionPage;

import pages.HomePage;

import pages.PurchasePage;


public class EmptyCardNumberTest extends BaseTest
{
    @Test

    public void verifyBookingWithEmptyCardNumber()
    {
        HomePage homePage =
            new HomePage(driver);

        homePage.searchFlight(
            "Paris",
            "London");

        FlightSelectionPage flightPage =
            new FlightSelectionPage(driver);

        flightPage.chooseFirstFlight();

        PurchasePage purchasePage =
            new PurchasePage(driver);
```

```
purchasePage.enterPassengerDetails(  
    "Pranshu",  
    "ABC Street",  
    "Varanasi",  
    "UP",  
    "221001");  
  
purchasePage.enterPaymentDetails(  
    "Visa",  
    "",  
    "11",  
    "2028",  
    "Pranshu Mishra");  
purchasePage.clickPurchaseFlight();  
ConfirmationPage confirmationPage =  
    new ConfirmationPage(driver);  
Assert.assertTrue(  
    confirmationPage  
        .getConfirmationMessage()  
        .contains("Thank you"));  
System.out.println(  
    "DEFECT FOUND : Booking allowed with empty card number");  
}  
}
```

5.2 EmptyPassengerDetailsTest.java

```
package defecttests;

import org.testng.Assert;

import org.testng.annotations.Test;

import base.BaseTest;

import pages.ConfirmationPage;

import pages.FlightSelectionPage;

import pages.HomePage;

import pages.PurchasePage;


public class EmptyPassengerDetailsTest extends BaseTest
{
    @Test

    public void verifyBookingWithEmptyPassengerDetails()
    {

        HomePage homePage =

            new HomePage(driver);

        homePage.searchFlight(

            "Paris",

            "Cairo");

        FlightSelectionPage flightPage =

            new FlightSelectionPage(driver);

        flightPage.chooseFirstFlight();

        PurchasePage purchasePage =

            new PurchasePage(driver);
```

```

purchasePage.enterPassengerDetails(
    "",
    "",
    "",
    "",
    "");

purchasePage.enterPaymentDetails(
    "Visa",
    "4111111111111111",
    "11",
    "2028",
    "Test User");

purchasePage.clickPurchaseFlight();

ConfirmationPage confirmationPage =
    new ConfirmationPage(driver);

Assert.assertTrue(
    confirmationPage
        .getConfirmationMessage()
        .contains("Thank you"));

System.out.println(
    "DEFECT FOUND : Booking allowed with empty passenger details");
}
}

```

5.3 ExpiredCardTest.java

```
package defecttests;

import org.testng.Assert;

import org.testng.annotations.Test;

import base.BaseTest;

import pages.ConfirmationPage;

import pages.FlightSelectionPage;

import pages.HomePage;

import pages.PurchasePage;


public class ExpiredCardTest extends BaseTest
{
    @Test

    public void verifyBookingWithExpiredCard()
    {

        HomePage homePage =

            new HomePage(driver);

        homePage.searchFlight(

            "Paris",

            "London");


        FlightSelectionPage flightPage =

            new FlightSelectionPage(driver);

        flightPage.chooseFirstFlight();

        PurchasePage purchasePage =

            new PurchasePage(driver);
```

```
purchasePage.enterPassengerDetails(

    "Pranshu",

    "ABC Street",

    "Varanasi",

    "UP",

    "221001");


purchasePage.enterPaymentDetails(

    "Visa",

    "4111111111111111",

    "01",

    "2018",

    "Pranshu Mishra");


purchasePage.clickPurchaseFlight();

ConfirmationPage confirmationPage =

    new ConfirmationPage(driver);

Assert.assertTrue(

    confirmationPage

        .getConfirmationMessage()

        .contains("Thank you"));

System.out.println(

    "DEFECT FOUND : Booking allowed with expired card");

}

}
```

6. Listeners Package

6.1 TestListener.java

```
package listeners;

import org.testng.ITestContext;

import org.testng.ITestListener;

import org.testng.ITestResult;

import com.aventstack.extentreports.ExtentReports;

import com.aventstack.extentreports.ExtentTest;

import utilities.DriverFactory;

import utilities.ExtentManager;

import utilities.ScreenshotUtil;

public class TestListener implements ITestListener
{

    private static ExtentReports extentReports =

        ExtentManager.getInstance();

    private static ExtentTest extentTest;

    @Override

    public void onTestStart(ITestResult result)

    {
```

```
extentTest =

    extentReports.createTest(

        result.getName());

System.out.println(

    "STARTED : "

        + result.getName());
}

@Override

public void onTestSuccess(ITestResult result)

{

    extentTest.pass(

        "Test Passed");

System.out.println(

    "PASSED : "

        + result.getName());
}

@Override

public void onTestFailure(ITestResult result)

{

    String screenshotPath =
```

```
        ScreenshotUtil.captureScreenshot(

            DriverFactory.getDriver(),

            result.getName());

    extentTest.fail(result.getThrowable());

    extentTest.addScreenCaptureFromPath(

        screenshotPath);
}

@Override

public void onStart(ITestContext context)

{

    System.out.println(

        "TEST EXECUTION STARTED");
}

@Override

public void onFinish(ITestContext context)

{

    extentReports.flush();

    System.out.println(

        "TEST EXECUTION FINISHED");
}
}
```

7. Pages Package

7.1 HomePage.java

```
package pages;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.Select;

import utilities.WaitUtils;


public class HomePage

{

    WebDriver driver;

    By fromPort = By.name("fromPort");

    By toPort = By.name("toPort");

    By findFlightsButton = By.cssSelector("input[type='submit']");

    public HomePage(WebDriver driver)

    {

        this.driver = driver;

    }


    public void searchFlight(String fromCity, String toCity)

    {

        WebElement fromDropdown =

            WaitUtils.waitForElementVisible(driver, fromPort);
```

```

        Select from = new Select(fromDropdown);

        from.selectByVisibleText(fromCity);


        WebElement toDropdown =

            WaitUtils.waitForElementVisible(driver, toPort);


        Select to = new Select(toDropdown);

        to.selectByVisibleText(toCity);


        WaitUtils.waitForElementVisible(driver, findFlightsButton)

            .click();
    }
}

```

7.2 FlightSelectionPage.java

```

package pages;


import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class FlightSelectionPage
{
    WebDriver driver;

    public FlightSelectionPage(WebDriver driver)
    {
        this.driver = driver;
    }
}

```

```
By flightTable = By.className("table");
```

```
By chooseFlightButton =
```

```
By.xpath("//input[@type='submit']")[1];
```

```
public boolean isFlightListDisplayed()
```

```
{
```

```
    return driver.findElement(
```

```
        flightTable)
```

```
        .isDisplayed();
```

```
}
```

```
public void chooseFirstFlight()
```

```
{
```

```
    driver.findElement(
```

```
        chooseFlightButton)
```

```
        .click();
```

```
}
```

```
}
```

7.3 PurchasePage.java

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.support.ui.Select;

import utilities.WaitUtils;

public class PurchasePage {

    WebDriver driver;

    By purchasePageTitle = By.tagName("h2");

    By airlineInfo =

        By.xpath("//p[contains(text(),'Airline')]");

    By totalCostInfo =

        By.xpath("//p[contains(text(),'Price')]");

    By nameField = By.id("inputName");

    By addressField = By.id("address");

    By cityField = By.id("city");

    By stateField = By.id("state");

    By zipCodeField = By.id("zipCode");
```

By cardTypeDropdown =

By.id("cardType");

By creditCardNumber =

By.id("creditCardNumber");

By creditCardMonth =

By.id("creditCardMonth");

By creditCardYear =

By.id("creditCardYear");

By nameOnCard =

By.id("nameOnCard");

By purchaseFlightButton =

By.cssSelector("input[type='submit']");

public PurchasePage(WebDriver driver)

{

 this.driver = driver;

}

public String getPurchasePageTitle()

{

```
return WaitUtils  
    .waitForElementVisible(  
        driver,  
        purchasePageTitle)  
    .getText();  
}
```

```
public boolean isAirlineDisplayed()  
{
```

```
    return driver.findElement(  
        airlineInfo)  
        .isDisplayed();  
}
```

```
public boolean isTotalCostDisplayed()  
{
```

```
    return driver.findElement(  
        totalCostInfo)  
        .isDisplayed();  
}
```

```
public void enterPassengerDetails(  
    String name,  
    String address,
```

```
String city,  
String state,  
String zipCode) {
```

```
WaitUtils.waitForElementVisible(  
    driver,  
    nameField)  
    .sendKeys(name);
```

```
WaitUtils.waitForElementVisible(  
    driver,  
    addressField)  
    .sendKeys(address);
```

```
WaitUtils.waitForElementVisible(  
    driver,  
    cityField)  
    .sendKeys(city);
```

```
WaitUtils.waitForElementVisible(  
    driver,  
    stateField)  
    .sendKeys(state);
```

```
WaitUtils.waitForElementVisible(  
    driver,  
    zipCodeField)
```

```
        .sendKeys(zipCode);  
    }  
}
```

```
public void enterPaymentDetails(  
    String cardType,  
    String cardNumber,  
    String month,  
    String year,  
    String cardHolderName)  
{
```

```
    Select cardDropdown =  
        new Select(  
            driver.findElement(  
                cardTypeDropdown));
```

```
    cardDropdown.selectByVisibleText(  
        cardType);
```

```
    driver.findElement(  
        creditCardNumber)  
        .sendKeys(cardNumber);
```

```
    driver.findElement(  
        creditCardMonth)  
        .clear();
```

```
driver.findElement(  
    creditCardMonth)  
    .sendKeys(month);
```

```
driver.findElement(  
    creditCardYear)  
    .clear();
```

```
driver.findElement(  
    creditCardYear)  
    .sendKeys(year);
```

```
driver.findElement(  
    nameOnCard)  
    .sendKeys(cardHolderName);  
}
```

```
public void clickPurchaseFlight()  
{
```

```
    driver.findElement(  
        purchaseFlightButton)  
        .click();  
    }  
}
```

7.4 ConfirmationPage.java

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

import utilities.WaitUtils;

public class ConfirmationPage
{

    WebDriver driver;

    By confirmationMessage =
        By.tagName("h1");

    By bookingId =
        By.xpath("//td[text()='Id']/following-sibling::td");

    By bookingStatus =
        By.xpath("//td[text()='Status']/following-sibling::td");

    By bookingAmount =
        By.xpath("//td[text()='Amount']/following-sibling::td");

    public ConfirmationPage(
```

```
WebDriver driver){

    this.driver = driver;
}

public String getConfirmationMessage()
{

    return WaitUtils

        .waitForElementVisible(

            driver,

            confirmationMessage)

        .getText();
}

public String getBookingId()
{

    return driver

        .findElement(bookingId)

        .getText();
}

public String getBookingStatus()
{

    return driver
```

```
        .findElement(bookingStatus)

        .getText();
    }

    public String getBookingAmount()
    {

        return driver

            .findElement(bookingAmount)

            .getText();
    }
}
```

8. Test Cases Package

8.1 LaunchTest.java

```
package testcases;

import org.testng.Assert;

import org.testng.annotations.Test;

import base.BaseTest;
```

```

public class LaunchTest extends BaseTest
{
    @Test

    public void verifyApplicationLaunch()
    {
        String actualTitle = driver.getTitle();

        System.out.println("Page Title: " + actualTitle);

        Assert.assertFalse(actualTitle.isEmpty());
    }
}

```

8.2 SearchFlightTest.java

```

package testcases;

import org.testng.Assert;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.HomePage;

public class SearchFlightTest extends BaseTest
{
    @Test

    public void verifyFlightSearch()
    {
        HomePage homePage = new HomePage(driver);

        homePage.searchFlight("Paris","London");

        String currentUrl = driver.getCurrentUrl();

        System.out.println(currentUrl);

        Assert.assertTrue(currentUrl.contains("reserve"));
    }
}

```

8.3 EndToEndBookingTest.javapackage testcases;

```
import org.testng.Assert;

import org.testng.annotations.Test;

import base.BaseTest;

import dataproviders.BookingDataProvider;

import pages.ConfirmationPage;

import pages.FlightSelectionPage;

import pages.HomePage;

import pages.PurchasePage;


public class EndToEndBookingTest extends BaseTest
{
    @Test(
        dataProvider = "passengerData",
        dataProviderClass = BookingDataProvider.class)
    public void verifyCompleteBookingFlow(
        String fromCity,
        String toCity,
        String name,
        String address,
        String city,
        String state,
        String zipCode,
        String cardType,
        String cardNumber,
```

```
String month,

String year,

String nameOnCard) {

HomePage homePage =

    new HomePage(driver);

homePage.searchFlight(fromCity, toCity);


FlightSelectionPage flightPage =

    new FlightSelectionPage(driver);


Assert.assertTrue(

    flightPage.isFlightListDisplayed(),

    "Flight list is not displayed");

flightPage.chooseFirstFlight();

PurchasePage purchasePage =

    new PurchasePage(driver);


String purchaseTitle =

    purchasePage.getPurchasePageTitle();

System.out.println(

    "Purchase Page Title : "

    + purchaseTitle);

Assert.assertTrue(

    purchaseTitle.contains(

        "Your flight"),

    "Purchase Flight page not displayed");
```

```
Assert.assertTrue(  
    purchasePage.isAirlineDisplayed(),  
    "Airline information not displayed");
```

```
Assert.assertTrue(  
    purchasePage.isTotalCostDisplayed(),  
    "Price information not displayed");
```

```
purchasePage.enterPassengerDetails(  
    name,  
    address,  
    city,  
    state,  
    zipCode);
```

```
purchasePage.enterPaymentDetails(  
    cardType,  
    cardNumber,  
    month,  
    year,  
    nameOnCard);
```

```
purchasePage.clickPurchaseFlight();
```

```
ConfirmationPage confirmationPage =  
    new ConfirmationPage(driver);
```

```
String actualMessage =  
    confirmationPage.getConfirmationMessage();
```

```
String bookingId =  
    confirmationPage.getBookingId();
```

```
String bookingStatus =  
    confirmationPage.getBookingStatus();
```

```
String bookingAmount =  
    confirmationPage.getBookingAmount();
```

```
System.out.println(  
    "Route : "  
        + fromCity  
        + " -> "  
        + toCity);
```

```
System.out.println(  
    "Passenger : "  
        + name);
```

```
System.out.println(  
    "Confirmation Message : "  
        + actualMessage);
```

```
System.out.println(
```

```
        "Booking ID : "
        + bookingId);

System.out.println(

    "Booking Status : "
        + bookingStatus);

System.out.println(

    "Booking Amount : "
        + bookingAmount);

Assert.assertTrue(

    actualMessage.contains(

        "Thank you"),

    "Booking confirmation failed");

Assert.assertFalse(

    bookingId.isEmpty(),

    "Booking ID not generated");

Assert.assertEquals(

    bookingStatus,

    "PendingCapture",

    "Incorrect booking status");

Assert.assertTrue(

    bookingAmount.contains(

        "USD"),

    "Booking amount not displayed");

    }

}
```

8.4 CrossBrowserLaunchTest.java

```
package testcases;

import org.testng.Assert;

import org.testng.annotations.Test;

import base.BaseTest;


public class CrossBrowserLaunchTest extends BaseTest

{

    @Test

    public void verifyApplicationLaunchInDifferentBrowsers()

    {

        String actualTitle = driver.getTitle();

        System.out.println("Browser Title: " + actualTitle);

        Assert.assertEquals(actualTitle, "BlazeDemo");

    }

}
```

9. Utilities Package

9.1 ConfigReader.java

```
package utilities;

import java.io.FileInputStream;

import java.io.IOException;

import java.util.Properties;
```

```
public class ConfigReader
{
    static Properties properties;

    static
    {
        try
        {
            FileInputStream file =
                new FileInputStream(
                    "src/test/resources/config.properties");

            properties = new Properties();
            properties.load(file);
        }
        catch (IOException e)
        {

            e.printStackTrace();
        }
    }

    public static String getProperty(String key)
    {

        return properties.getProperty(key);
    }
}
```

9.2 DriverFactory.java

```
package utilities;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.chrome.ChromeDriver;

import org.openqa.selenium.edge.EdgeDriver;

import org.openqa.selenium.firefox.FirefoxDriver;

import org.openqa.selenium.firefox.FirefoxOptions;


import io.github.bonigarcia.wdm.WebDriverManager;


public class DriverFactory
{

    private static WebDriver driver;


    public static WebDriver initializeDriver(String browser)
    {

        if (browser.equalsIgnoreCase("chrome"))
        {

            WebDriverManager.chromedriver().setup();

            driver = new ChromeDriver();

        }

        else if (browser.equalsIgnoreCase("firefox"))
        {
```

```

        WebDriverManager.firefoxdriver().setup();

        FirefoxOptions options = new FirefoxOptions();

        driver = new FirefoxDriver(options);
    }

    else if (browser.equalsIgnoreCase("edge"))
    {

        System.setProperty(

            "webdriver.edge.driver",

            System.getProperty("user.dir") + "\\drivers\\msedgedriver.exe"

        );

        driver = new EdgeDriver();
    }

    else
    {

        throw new RuntimeException(

            "Unsupported Browser : " + browser

        );
    }

    driver.manage().window().maximize();

    return driver;
}

public static WebDriver getDriver()
{

    return driver;
}
}

```

9.3 ExcelUtils.java

```
package utilities;

import java.io.FileInputStream;

import java.io.IOException;

import org.apache.poi.ss.usermodel.DataFormatter;

import org.apache.poi.xssf.usermodel.XSSFSheet;

import org.apache.poi.xssf.usermodel.XSSFWorkbook;


public class ExcelUtils

{

    public static Object[][] getTestData(String sheetName)

    {

        Object[][] data = null;

        try {

            FileInputStream file =

                new FileInputStream(

                    "testdata/BookingData.xlsx");

            XSSFWorkbook workbook =

                new XSSFWorkbook(file);

            XSSFSheet sheet =

                workbook.getSheet(sheetName);

            int rowCount =
```

```

        sheet.getPhysicalNumberOfRows();

    int colCount =

        sheet.getRow(0)

            .getPhysicalNumberOfCells();

    data =

        new Object[rowCount - 1][colCount];

    DataFormatter formatter =

        new DataFormatter();

    for (int i = 1; i < rowCount; i++)

    {

        for (int j = 0; j < colCount; j++)

        {

            data[i - 1][j] =

                formatter.formatCellValue(

                    sheet.getRow(i)

                        .getCell(j));

        }

    }

    workbook.close();

}

catch (IOException e)

{

    e.printStackTrace();

}

return data;

}

}

```

9.4 ExtentManager.java

```
package utilities;

import com.aventstack.extentreports.ExtentReports;

import com.aventstack.extentreports.reporter.ExtentSparkReporter;


public class ExtentManager
{

    private static ExtentReports extentReports;

    public static ExtentReports getInstance()
    {

        if (extentReports == null) {

            ExtentSparkReporter sparkReporter =

                new ExtentSparkReporter(

                    "reports/ExtentReport.html");

            sparkReporter.config()

                .setReportName(

                    "BlazeDemo Automation Report");

            sparkReporter.config()

                .setDocumentTitle(

                    "Automation Execution Report");

            extentReports =

                new ExtentReports();

            extentReports.attachReporter(

                sparkReporter);

            extentReports.setSystemInfo(

                "Framework",
```

```

        "Selenium TestNG");

    extentReports.setSystemInfo(

        "Application",

        "BlazeDemo");

    }

    return extentReports;

}

}

```

9.5 ScreenshotUtil.java

```

package utilities;

import java.io.File;

import java.io.IOException;

import java.nio.file.Files;

import java.nio.file.StandardCopyOption;

import java.time.LocalDateTime;

import java.time.format.DateTimeFormatter;

import org.openqa.selenium.OutputType;

import org.openqa.selenium.TakesScreenshot;

import org.openqa.selenium.WebDriver;

public class ScreenshotUtil

{

    public static String captureScreenshot(

        WebDriver driver,

        String testName) {

        String timestamp =

```

```

        LocalDateTime.now()

            .format(DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss"));

String filePath =

    "screenshots/"

        + testName

        + " _ "

        + timestamp

        + ".png";

File source =

    ((TakesScreenshot) driver)

        .getScreenshotAs(OutputType.FILE);

File destination =

    new File(filePath);

try

{

    Files.copy(

        source.toPath(),

        destination.toPath(),

        StandardCopyOption.REPLACE_EXISTING);

}

catch (IOException e)

{

    e.printStackTrace();

}

return filePath;

}

}

```

9.6 WaitUtils.java

```
package utilities;

import java.time.Duration;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.WebDriverWait;


public class WaitUtils

{

    public static WebElement waitForElementVisible(

        WebDriver driver,

        By locator)

    {

        WebDriverWait wait =

            new WebDriverWait(driver,

                Duration.ofSeconds(10));

        return wait.until(

            ExpectedConditions

                .visibilityOfElementLocated(locator));

    }

}
```

10. Resources

10.1 config.properties

url=<https://blazedemo.com/>
browser=[chrome](#)

11. Test Data

11.1 BookingData.xlsx

FromCity	ToCity	Name	Address	City	State	ZipCode	CardType	CardNumber	Month	Year	NameOn Card
Paris	London	Pranshu	ABC Street	Varanasi	UP	221001	Visa	41111111111111110	12	2028	Pranshu Mishra
Boston	Rome	Rahul	XYZ Colony	Delhi	Delhi	110001	American Express	378282246310005	11	2027	Rahul Sharma
Mexico City	Berlin	Aman	MG Road	Bangalore	Karnataka	560001	Diner's Club	30569309025904	10	2029	Aman Verma

12. TestNG Suite Files

12.1 testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="BlazeDemo Suite">

  <test name="Chrome Test">

    <parameter name="browser"
      value="chrome"/>

    <classes>
      <class name="testcases.LaunchTest"/>
      <class name="testcases.EndToEndBookingTest"/>
      <class name="testcases.SearchFlightTest"/>
    </classes>

  </test>

</suite>
```

12.2 crossbrowser.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Cross Browser Suite">

  <test name="Chrome Test">
    <parameter name="browser" value="chrome"/>

    <classes>
      <class name="testcases.CrossBrowserLaunchTest"/>
    </classes>
  </test>

  <test name="Firefox Test">
    <parameter name="browser" value="firefox"/>

    <classes>
      <class name="testcases.CrossBrowserLaunchTest"/>
    </classes>
  </test>

  <test name="Edge Test">
    <parameter name="browser" value="edge"/>

    <classes>
      <class name="testcases.CrossBrowserLaunchTest"/>
    </classes>
  </test>

</suite>
```

12.3 defect-testng.xml

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="BlazeDemo Defect Suite">

  <test name="Defect Tests">

    <parameter name="browser"
      value="chrome"/>

    <classes>

      <class name="defecttests.EmptyPassengerDetailsTest"/>

      <class name="defecttests.EmptyCardNumberTest"/>

    </classes>

  </test>

</suite>
```

```
<class name="defecttests.ExpiredCardTest"/>

</classes>

</test>

</suite>
```

13. Maven Configuration

13.1 pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance" xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>BlazeDemoAutomation</groupId>
  <artifactId>BlazeDemoAutomation</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
  </properties>

  <dependencies>

    <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.34.0</version>
    </dependency>

    <dependency>
      <groupId>org.testng</groupId>
      <artifactId>testng</artifactId>
      <version>7.11.0</version>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>io.github.bonigarcia</groupId>
      <artifactId>webdrivermanager</artifactId>
      <version>6.3.2</version>
    </dependency>

    <dependency>
      <groupId>org.apache.poi</groupId>
```

```
<artifactId>poi-ooxml</artifactId>
<version>5.4.1</version>
</dependency>

<dependency>
<groupId>com.aventstack</groupId>
<artifactId>extentreports</artifactId>
<version>5.1.2</version>
</dependency>

    <dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-api</artifactId>
<version>2.25.1</version>
</dependency>

<dependency>
<groupId>org.apache.logging.log4j</groupId>
<artifactId>log4j-core</artifactId>
<version>2.25.1</version>
</dependency>

</dependencies>

<build>

<plugins>

<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>3.5.3</version>

<configuration>
<suiteXmlFiles>
<suiteXmlFile>testng.xml</suiteXmlFile>
</suiteXmlFiles>
</configuration>

</plugin>

</plugins>
</build>
</project>
```

14. Jenkins Integration

14.1 Jenkinsfile

```
pipeline {  
  
    agent any  
  
    tools {  
        jdk 'JDK'  
        maven 'Maven3'  
    }  
  
    stages {  
  
        stage('Checkout Code'){  
            steps {  
                git branch: 'main', url: 'https://github.com/PRANSHU9999/Capstone_Project.git'  
            }  
        }  
  
        stage('Clean Project'){  
            steps {  
                bat 'mvn clean'  
            }  
        }  
  
        stage('Compile Project'){  
            steps {  
                bat 'mvn compile'  
            }  
        }  
  
        stage('Execute TestNG Tests'){  
            steps {  
                bat 'mvn test'  
            }  
        }  
  
        stage('Package Project'){  
            steps {  
                bat 'mvn package'  
            }  
        }  
  
        stage('Archive Reports'){  
            steps {  
  
                archiveArtifacts(  
                    artifacts: 'reports/**/*.*',  
                    allowEmptyArchive: true  
                )  
  
                archiveArtifacts(  

```

```
    artifacts: 'screenshots/**/*.*',
    allowEmptyArchive: true
  )

  archiveArtifacts(
    artifacts: 'test-output/**/*.*',
    allowEmptyArchive: true
  )
}
}
}

post {
  success {
    echo 'BlazeDemo Automation Build Successful'
  }

  failure {
    echo 'BlazeDemo Automation Build Failed'
  }

  always {
    echo 'Pipeline Execution Completed'
  }
}
}
```
